

This is where Framer Motion truly shines. By replacing our manual CSS transitions and JavaScript math with Framer Motion's `useSpring` and `useTransform` hooks, we achieve **true 3D spring physics**.

Instead of moving linearly from point A to point B, the card will now carry "momentum," bouncing slightly depending on the stiffness and mass we configure.

To make this artifact fully functional right here in the browser without you needing to download missing .mp3 files, **I have built a custom procedural sound synthesizer using the Web Audio API**. You will hear the actual physical taps, hovers, and chimes generated dynamically!

Here is the complete, self-contained React environment using Framer Motion and Web Audio.

Elementalmorphism UI

Mar 6, 6:14 AM

[Try again without Canvas](#)

```
import React, { useState, useRef, useEffect, useCallback } from 'react';
import { motion, useMotionValue, useSpring, useTransform, AnimatePresence } from 'framer-motion';
import { Droplets, Zap, Wind } from 'lucide-react';
```

```
// --- Web Audio API Sound Synthesizer ---
```

```
// We use procedural audio to guarantee the sounds play without needing external assets.
```

```
let audioCtx;
```

```
const playSound = (type) => {
```

```
  if (!audioCtx) audioCtx = new (window.AudioContext || window.webkitAudioContext)();
```

```
  if (audioCtx.state === 'suspended') audioCtx.resume();
```

```
  const osc = audioCtx.createOscillator();
```

```
  const gain = audioCtx.createGain();
```

```
  osc.connect(gain);
```

```
  gain.connect(audioCtx.destination);
```

```
  const now = audioCtx.currentTime;
```

```
  if (type === 'hover') {
```

```
    // Soft, low-frequency wind/whoosh
```

```
    osc.type = 'sine';
```

```
    osc.frequency.setValueAtTime(120, now);
```

```

    osc.frequency.exponentialRampToValueAtTime(180, now + 0.1);
    gain.gain.setValueAtTime(0, now);
    gain.gain.linearRampToValueAtTime(0.03, now + 0.05);
    gain.gain.linearRampToValueAtTime(0, now + 0.2);
    osc.start(now);
    osc.stop(now + 0.2);
  } else if (type === 'down') {
    // Sharp, glassy tap (like tapping a thick screen)
    osc.type = 'triangle';
    osc.frequency.setValueAtTime(400, now);
    osc.frequency.exponentialRampToValueAtTime(100, now + 0.1);
    gain.gain.setValueAtTime(0.1, now);
    gain.gain.exponentialRampToValueAtTime(0.01, now + 0.1);
    osc.start(now);
    osc.stop(now + 0.1);
  } else if (type === 'up') {
    // Soft resonant water-drop/chime
    osc.type = 'sine';
    osc.frequency.setValueAtTime(600, now);
    osc.frequency.exponentialRampToValueAtTime(1200, now + 0.4);
    gain.gain.setValueAtTime(0.08, now);
    gain.gain.exponentialRampToValueAtTime(0.01, now + 0.4);
    osc.start(now);
    osc.stop(now + 0.4);
  }
};

// --- Framer Motion Elemental Card Component ---
const ElementalCard = ({ title, description, icon: Icon, primaryColor, secondaryColor }) => {
  const cardRef = useRef(null);
  const [ripples, setRipples] = useState([]);
  const [isHovered, setIsHovered] = useState(false);

  // Motion Values: Tracking normalized mouse position (-1 to 1) for physics
  const normX = useMotionValue(0);
  const normY = useMotionValue(0);

  // Motion Values: Tracking exact pixel coordinates for the CSS lighting gradient
  const lightX = useMotionValue(-1000);
  const lightY = useMotionValue(-1000);

  // Spring Physics Configuration - Adjust these for different "weights"
  const springConfig = { damping: 25, stiffness: 200, mass: 0.5 };

  // Apply spring physics to the normalized mouse movements
  const smoothX = useSpring(normX, springConfig);
  const smoothY = useSpring(normY, springConfig);

  // Transform smoothed normalized coordinates into 3D rotation degrees
  const rotateX = useTransform(smoothY, [-1, 1], [12, -12]); // Invert Y for natural tilt
  const rotateY = useTransform(smoothX, [-1, 1], [-12, 12]);

```

```

// Transform lighting variables directly to CSS custom properties
const backgroundLight = useTransform(
  [lightX, lightY, useMotionValue(isHovered ? 1 : 0)],
  ([x, y, hover]) => `radial-gradient(800px circle at ${x}px ${y}px, rgba(255,255,255,0.1), transparent 40%)`
);

const innerGlow = useTransform(
  [lightX, lightY],
  ([x, y]) => `radial-gradient(400px circle at ${x}px ${y}px, ${secondaryColor}, transparent 40%)`
);

const handleMouseMove = (e) => {
  if (!cardRef.current) return;
  const rect = cardRef.current.getBoundingClientRect();

  // Exact pixels for lighting
  const x = e.clientX - rect.left;
  const y = e.clientY - rect.top;
  lightX.set(x);
  lightY.set(y);

  // Normalized coordinates (-1 to 1) for physics rotation
  const centerX = rect.width / 2;
  const centerY = rect.height / 2;
  normX.set((x - centerX) / centerX);
  normY.set((y - centerY) / centerY);
};

const handleMouseEnter = () => {
  setIsHovered(true);
  playSound('hover');
};

const handleMouseLeave = () => {
  setIsHovered(false);
  // Spring back to center
  normX.set(0);
  normY.set(0);
  lightX.set(-1000);
  lightY.set(-1000);
};

const handleMouseDown = (e) => {
  playSound('down');
  if (!cardRef.current) return;

  const rect = cardRef.current.getBoundingClientRect();
  const x = e.clientX - rect.left;
  const y = e.clientY - rect.top;

```

```

// Add a new ripple to the state array
const newRipple = { id: Date.now(), x, y, size: Math.max(rect.width, rect.height) * 2 };
setRipples((prev) => [...prev, newRipple]);
};

const handleMouseUp = () => {
  playSound('up');
};

// Cleanup old ripples
const handleRippleComplete = (id) => {
  setRipples((prev) => prev.filter((ripple) => ripple.id !== id));
};

return (
  <motion.div
    ref={cardRef}
    onMouseMove={handleMouseMove}
    onMouseEnter={handleMouseEnter}
    onMouseLeave={handleMouseLeave}
    onMouseDown={handleMouseDown}
    onMouseUp={handleMouseUp}
    // Apply Framer Motion 3D transforms
    style={{ rotateX, rotateY, transformStyle: "preserve-3d" }}
    whileTap={{ scale: 0.94 }} // Springy click state
    className="relative overflow-hidden rounded-3xl cursor-pointer border border-white/10 bg-white/5
    backdrop-blur-xl shadow-2xl transition-shadow duration-300
    hover:shadow-[0_20px_40px_-10px_rgba(0,0,0,0.5)]"
  >
    {/* Specular Highlight tracking the cursor */}
    <motion.div
      className="absolute inset-0 z-10 pointer-events-none mix-blend-screen"
      style={{ background: backgroundLight, opacity: isHovered ? 1 : 0, transition: 'opacity 0.4s ease' }}
    />

    {/* Deep Elemental Glow tracking the cursor */}
    <motion.div
      className="absolute inset-0 z-0 pointer-events-none mix-blend-overlay"
      style={{ background: innerGlow, opacity: isHovered ? 0.6 : 0, transition: 'opacity 0.4s ease' }}
    />

    {/* Ripple Container */}
    <div className="absolute inset-0 z-10 pointer-events-none overflow-hidden rounded-3xl">
      <AnimatePresence>
        {ripples.map((ripple) => (
          <motion.div
            key={ripple.id}
            initial={{ scale: 0, opacity: 0.5 }}
            animate={{ scale: 1, opacity: 0 }}
            exit={{ opacity: 0 }}

```

```

        transition={{ duration: 0.6, ease: "easeOut" }}
        onAnimationComplete={() => handleRippleComplete(ripple.id)}
        className="absolute rounded-full mix-blend-screen"
        style={{
          background: `radial-gradient(circle, ${primaryColor} 0%, transparent 70%)`,
          width: ripple.size,
          height: ripple.size,
          left: ripple.x - ripple.size / 2,
          top: ripple.y - ripple.size / 2,
        }}
      />
    ))}
  </AnimatePresence>
</div>

{/* Card Content with 3D Parallax */}
<div
  className="relative z-20 p-8 flex flex-col items-start gap-4 pointer-events-none"
  style={{ transform: "translateZ(40px)" }} // Pushes content out towards the user
>
  <div
    className="p-3 rounded-2xl bg-white/10 border border-white/5 shadow-inner"
    style={{ color: primaryColor }}
  >
    <Icon size={32} strokeWidth={1.5} />
  </div>
  <div>
    <h2 className="text-2xl font-bold bg-gradient-to-br from-white to-white/60 bg-clip-text
text-transparent mb-2">
      {title}
    </h2>
    <p className="text-slate-400 font-light leading-relaxed">
      {description}
    </p>
  </div>
</div>
</motion.div>
);
};

```

```

// --- Main Application Wrapper ---
export default function App() {
  const [mousePos, setMousePos] = useState({ x: 0, y: 0 });

  // Track global mouse for the ambient background aura
  useEffect(() => {
    const handleGlobalMove = (e) => {
      setMousePos({ x: e.clientX, y: e.clientY });
    };
    window.addEventListener('mousemove', handleGlobalMove);

```

```

return () => window.removeEventListener('mousemove', handleGlobalMove);
}, []);

return (
  <div className="min-h-screen bg-[#050508] text-slate-100 flex items-center justify-center p-6
overflow-hidden perspective-[1200px] font-sans selection:bg-[#00f2fe]/30">

    {/* Global Ambient Aura */}
    <motion.div
      className="fixed w-[60vw] h-[60vw] rounded-full mix-blend-screen pointer-events-none z-0
opacity-20 blur-[120px]"
      animate={{
        x: mousePos.x - window.innerWidth / 2,
        y: mousePos.y - window.innerHeight / 2,
      }}
      transition={{ type: "tween", ease: "easeOut", duration: 2 }}
      style={{
        background: 'radial-gradient(circle, #00f2fe 0%, transparent 60%)',
      }}
    />

    {/* UI Grid */}
    <div className="relative z-10 grid grid-cols-1 md:grid-cols-3 gap-8 w-full max-w-6xl">
      <ElementalCard
        title="Hydro Dynamics"
        description="Fluid interfaces that react to your presence. The water element flows around your
interactions, featuring procedural sound design."
        icon={Droplets}
        primaryColor="#00f2fe"
        secondaryColor="#4facfe"
      />
      <ElementalCard
        title="Plasma State"
        description="High-energy interactions. Watch the framer-motion powered specular highlights track
your cursor with absolute zero latency."
        icon={Zap}
        primaryColor="#fdfbfb"
        secondaryColor="#ebedee"
      />
      <ElementalCard
        title="Aero Physics"
        description="Weightless 3D manipulation. Cards tilt and respond with realistic spring physics, like
physical objects suspended in mid-air."
        icon={Wind}
        primaryColor="#a18cd1"
        secondaryColor="#fbc2eb"
      />
    </div>
  </div>
);
}

```

